Université Mohammed V de Rabat
École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes

ENSIAS

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Deep Learning – Lecture 9

Attention et Transformers

Pr. Aissam BERRAHOU

ENSIAS – Université Mohammed V de Rabat

Module Deep Learning – 2026-2027

Objectifs de la lecture

- expliquer les limites des RNN encodeur-décodeur et l'intuition de l'attention ;
- calculer à la main une attention par produit scalaire normalisé ;
- définir auto-attention, attention multi-têtes et masque causal ;
- expliquer l'encodage de position ;
- décrire un bloc Transformer et compter ses paramètres ;
- distinguer BERT, GPT, T5 et ViT ; implémenter un bloc avec Keras.

Plan de la lecture

1. Des RNN à l'attention

2. Attention par produit scalaire

3. Le Transformer

4. Familles de Transformers

Plan

1. Des RNN à l'attention

2. Attention par produit scalaire

3. Le Transformer

4. Familles de Transformers

Limites des RNN séquence à séquence

- **Goulot** : un seul vecteur de contexte pour toute la phrase source.
- **Séquentiel** : T pas successifs, peu parallélisable.
- **Distance** : l'information entre deux mots éloignés de k traverse k pas.

Attention (Bahdanau, 2015)

Pour chaque pas du décodeur : scores $e_{t,i}$ avec chaque état $\mathbf{h}_i$ de l'encodeur, poids $\alpha_{t,i} = \mathrm{softmax}_i(e_{t,i})$, contexte $\mathbf{c}_t = \sum_i \alpha_{t,i}\mathbf{h}_i$.

Alignement

« the European economic area » $\rightarrow$ « la zone économique européenne » : « zone » regarde « area », « européenne » regarde « European ».

Plan

1. Des RNN à l'attention

2. Attention par produit scalaire

3. Le Transformer

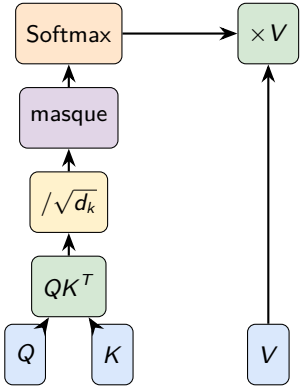
4. Familles de Transformers

Requêtes, clés, valeurs

Formule

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- recherche « floue » dans un dictionnaire ;
- $\sqrt{d_k}$: $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$; évite la saturation de la Softmax ;
- masque : $-\infty$ avant la Softmax.



Calcul à la main

$\mathbf{q} = (1, 0)$; $\mathbf{k}_1 = (1, 0)$, $\mathbf{k}_2 = (0, 1)$, $\mathbf{k}_3 = (1, 1)$; $v = (10, 20, 30)$; $d_k = 2$

Étapes

Scores $/\sqrt{2}$: (0,707; 0; 0,707)
exp : (2,03; 1; 2,03), somme 5,06
Poids : (0,40; 0,20; 0,40)
Sortie : $0{,}4 \cdot 10 + 0{,}2 \cdot 20 + 0{,}4 \cdot 30 = \mathbf{20}$

Les clés alignées avec la requête pèsent deux fois plus.

Auto-attention et multi-têtes

Auto-attention

$Q = XW^Q$, $K = XW^K$, $V = XW^V$: chaque élément se recompose à partir de toute la séquence.

Coréférence

« L'animal n'a pas traversé la rue car *il* était trop fatigué » : « il » $\rightarrow$ « animal » ; avec « large » : « il » $\rightarrow$ « rue ».

Multi-têtes

$\mathrm{MultiHead}(X) = [\mathrm{tête}_1; \ldots; \mathrm{tête}_h]W^O$, $d_k = d/h$: chaque tête une relation.

	Coût	**Séquentiel**	**Chemin**
Auto-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Récurrente	$O(nd^2)$	$O(n)$	$O(n)$

Masque causal et encodage de position

Masque causal

La position i ne voit que $j \leq i$:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Encodage de position

L'attention ignore l'ordre : « le chien mord l'homme » = « l'homme mord le chien ».

$PE_{(pos,2i)} = \sin \frac{pos}{10000^{2i/d}}$, $PE_{(pos,2i+1)} = \cos \frac{pos}{10000^{2i/d}}$

Variantes : appris (BERT, GPT), relatifs (RoPE).

Plan

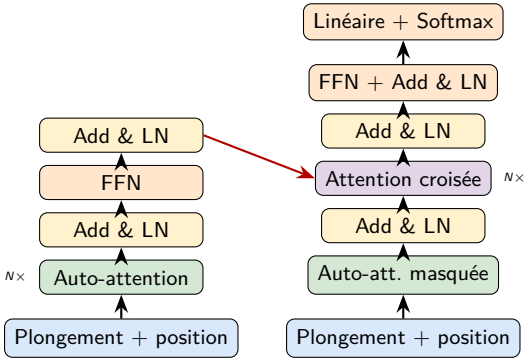
1. Des RNN à l'attention

2. Attention par produit scalaire

3. Le Transformer

4. Familles de Transformers

Architecture (Vaswani et al., 2017)



Un bloc

$\mathbf{z} = \mathrm{LN}(\mathbf{x} + \mathrm{MultiHead}(\mathbf{x}))$
$\mathbf{y} = \mathrm{LN}(\mathbf{z} + \mathrm{FFN}(\mathbf{z}))$
FFN par position, $d_{ff} = 4d$

Paramètres ($d = 512$, $d_{ff} = 2048$)

Attention $4d^2 + 4d \approx 1{,}05$ M
FFN $\approx 2{,}10$ M $\Rightarrow \approx 3{,}15$ M / bloc

Entraînement

Adam + échauffement, Dropout, *teacher forcing*, parallèle sur les positions.

Plan

1. Des RNN à l'attention

2. Attention par produit scalaire

3. Le Transformer

4. Familles de Transformers

BERT, GPT, T5, ViT

Famille	Structure	Pré-entraînement	Usages
BERT	encodeur	mots masqués (15 %)	classification, entités, QA
GPT	décodeur	mot suivant (causal)	génération, assistants
T5, BART	enc.-déc.	texte corrompu	traduction, résumé
ViT	encodeur	patchs 16×16	vision

Paradigme actuel

Pré-entraînement **auto-supervisé** massif $\rightarrow$ adaptation (fine-tuning ou consigne).

Coût quadratique

$n = 8\,000$: 64 M scores par tête et par couche $\Rightarrow$ attention éparse, fenêtrée, FlashAttention.

Un bloc Transformer dans Keras

```
class BlocTransformer(layers.Layer):
    def __init__(self, d, h, d_ff, taux=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(num_heads=h, key_dim=d // h)
        self.ffn = tf.keras.Sequential([layers.Dense(d_ff, activation='relu'), layers.Dense(d)])
        self.ln1, self.ln2 = layers.LayerNormalization(), layers.LayerNormalization()
        self.dr1, self.dr2 = layers.Dropout(taux), layers.Dropout(taux)
    def call(self, x, training=False):
        x = self.ln1(x + self.dr1(self.att(x, x), training=training))
        return self.ln2(x + self.dr2(self.ffn(x), training=training))

inputs = layers.Input(shape=(200,))
x = PlongementPosition(200, 20000, 64)(inputs) # jetons + positions apprises
x = BlocTransformer(64, h=4, d_ff=128)(x)
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(1, activation='sigmoid')(x)
```

À retenir

- Attention : accès direct, pondéré et appris à toute la séquence.
- Transformer = auto-attention multi-têtes + FFN + résiduels + LayerNorm + positions.
- Parallélisable ; base des LLM et des ViT.

Questions de révision

1. Refaire le calcul avec $\mathbf{q} = (0, 2)$. ($\approx 23{,}4$)
2. Combien de scores pour $n = 512$ et $n = 4\,096$?
3. BERT ou GPT pour l'analyse de sentiment ? pour générer du code ?